

Multi-agent Architecture

- 1. What is Multi-Agent Architecture?
- 2. Why does Multi-Agent Architecture matter?
- 3. What are Key Characteristics of Multi-Agent Architecture?
- 4. What are Core Components of Multi-Agent Architecture?

1. What is Multi-Agent Architecture?

Multi-Agent Architecture refers to a system design where multiple AI agents—each with a specific role, responsibility, or expertise—work collaboratively or independently to solve complex tasks that a single agent alone may not handle efficiently.

In simple terms, it's like building a team of specialized bots, each responsible for a part of the problem, just like departments in a company (e.g., sales bot, tech support bot, analytics bot).

Example of Multi-Agent Architecture in Live Production Support:

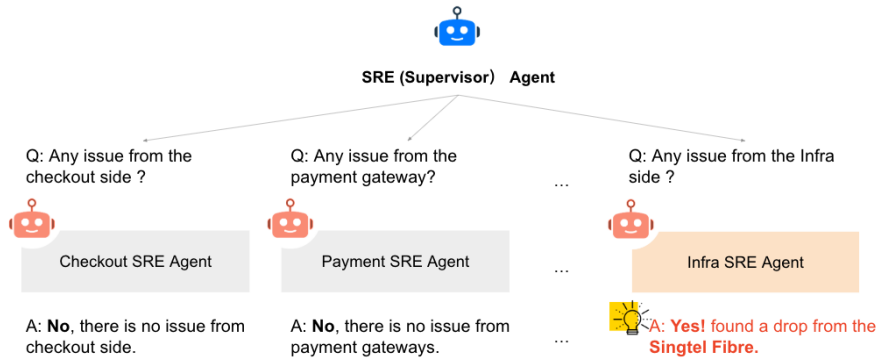
This example demonstrates a multi-agent system in action during a live production issue: a sudden drop in CCU in SG.

1. The **NOC Agent** detected the issue and notified the **Supervisor Agent**.
2. The **Supervisor Agent** initiated a root cause analysis by delegating the investigation to multiple specialized **SRE Agents** (e.g., Checkout, Payment, Infra).
3. Each **SRE Agent** independently checked its respective domain. The **Infra SRE Agent** identified the root cause — a drop in Singtel Fibre connectivity — and communicated the finding back to the **Supervisor Agent**.
4. The ISP-related failure was located within a minute, and the **Supervisor Agent** shared the insight to **SEER Agent**.
5. The **SEER Agent** was then tasked with generating a Confluence incident report, while another agent helped retrieve relevant monitoring data (e.g., Grafana links).

This showcases how multiple agents work collaboratively, in parallel and asynchronously, to accelerate detection, diagnosis, and resolution, a key strength of multi-agent architecture in real-time operational environments.

Live Production Support: multiple agents collaborate to solve the problem

Issue: CCU drop in SG → **Response:** Network of agents work together to find the root cause



Live Production Support: Auto detect issue, find root cause, generate incident report



2. Why does Multi-Agent Architecture matter?

Multi-agent architectures make AI systems more flexible, scalable, and easier to manage, especially in enterprise or complex workflows. You can update one agent without affecting the others and assign different teams to own different agents.

- **Efficiency:** Multi-AI agents enhance efficiency by distributing tasks among specialized agents. Each agent handles tasks it is best suited for. This ensures quicker and more accurate completion. The division of labor reduces bottlenecks and optimizes resource utilization, which leads to faster and more streamlined operations.
- **Flexibility:** Multi-AI agents are flexible enough to adapt to any changing requirements and environments. This is mostly due to each agent's independent nature. The flexibility allows businesses to respond swiftly to market changes and evolving customer needs.
- **Scalability:** Multi-AI agent systems are inherently scalable. New agents can be added as demand grows to handle increased workloads without disrupting the existing system. This scalability ensures that businesses can expand their operations and manage larger volumes of work efficiently.

3. What are Key Characteristics of Multi-Agent Architecture?

1. **Specialization:** Each agent is trained or prompted for a specific domain or function (e.g., data summarization, knowledge retrieval, user onboarding).
2. **Communication:** Agents can pass tasks or data to one another via tools, APIs, memory, or messaging protocols.
3. **Autonomy:** Agents operate semi-independently, often making decisions based on their role and inputs without needing central control.
4. **Coordination:** A "router agent" or orchestration layer often decides which agent should handle a given user request, enabling modular and scalable logic.

4. What are Core Components of Multi-Agent Architecture?

Component	Description
Agents	Each agent is an autonomous unit with: • A specific role or task (e.g., reasoning, search, summarization, data fetching) • Its own prompt, memory, and sometimes an embedded LLM • Decision-making capabilities based on its goals <i>Example: A “Research Agent” gathers data; a “Summary Agent” condenses it; a “Strategy Agent” proposes actions.</i>
Orchestrator / Router Agent	• Acts as a coordinator that routes tasks or messages to the appropriate agent. • Maintains the overall system logic, workflow, and task sequence. • Often performs intent classification or task decomposition.
Communication Protocol	• Defines how agents talk to each other—this could be via: • Message passing • Shared memory • Function calls with payloads (e.g., JSON) • Ensures inter-agent collaboration and handoff of tasks or context.
Shared Context / Memory	• Stores conversation history, task state, and relevant variables across agents. • Enables agents to build on each other’s outputs. • Can be implemented via: • Vector stores (for semantic memory) • Key-value stores (for parameters) • Session variables
Tool/API Layer	• Agents can call external tools, such as: • Databases • Calculators • APIs (CRM config values, JIRA ticket content, etc.) • Tool calling adds functionality beyond the LLM’s native abilities.
Knowledge Base	• A structured repository of static or retrievable content. • Shared among agents to ensure consistency in answers. • Often used for grounding responses or domain-specific rules.
Task Queue / Workflow Engine	• Manages multi-step workflows where agents take turns performing tasks. • Useful in more complex systems with long-running jobs or sequencing needs.
Logging and Monitoring	• Records agent decisions, tool outputs, errors, and response times. • Crucial for debugging and system transparency. • Helps refine agent performance over time.